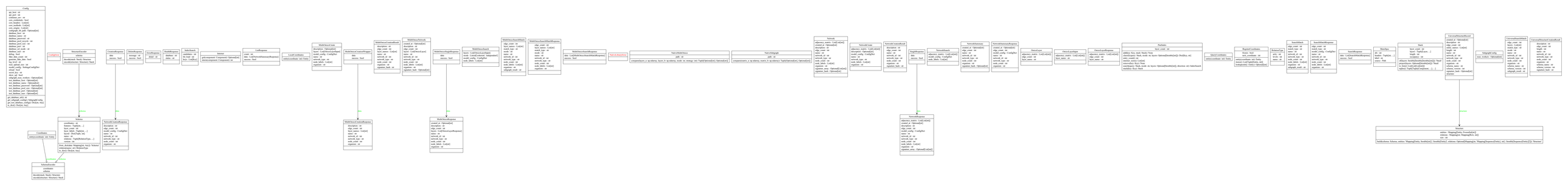

Config
api_host : str
api_port : int
container_env : str
cors_credentials : bool
cors_headers : List[str]
cors_methods : List[str]
cors_origins : List[str]
csubgraph_lib_path : Optional[str]
database_host : str
database_name : str
database_password : str
database_pool_recycle : int
database_pool_size : int
database_port : int
database_ssl_mode : str
database_user : str
debug : bool
fake_data_count : int
generate_fake_data : bool
log_level : str
max_upload_size : int
model_config : SettingsConfigDict
reload : bool
secret_key : str
show_sql : bool
subgraph_max_workers : Optional[int]
test_database_host : Optional[str]
test_database_name : Optional[str]
test_database_password : Optional[str]
test_database_pool_size : Optional[int]
test_database_port : Optional[int]
test_database_user : Optional[str]
get_database_url(): str
get_subgraph_config(): SubgraphConfig
get_test_database_config(): Dict[str, Any]
to_dict(): Dict[str, Any]
ConfigError
StructureEncoder
schema
decode(stack: Stack): Structure
encode(structure: Structure): Stack
schema
Schema
coordinates : str
features : Tuple[str, ...]
layer_count : int
layer_labels : Tuple[str, ...]
layout : Dict[Tuple, int]
name : str
relations : Tuple[RelationType, ...]
version : int
from_dict(data: Mapping[str, Any]): 'Schema'
relation(name: str): RelationType
to_dict(): Dict[str, Any]
Coordinates
entity(coordinate: int): Entity
coordinates
schema
SchemaEncoder
coordinates
schema
decode(stack: Stack): Structure
encode(structure: Structure): Stack
CreationResponse
data
success : bool
data
NetworkCreationResponse
description : str
edge_count : int
model_config : ConfigDict
name : str
network_id : int
network_type : str
node_count : int
organism : str
DeleteResponse
message : str
success : bool
ErrorResponse
detail : str
HealthResponse
database : str
status : str
IndexSearch
candidates : int
hit_load : int
keys : List[Key]
Interner
get(component: Component): Optional[int]
intern(component: Component): int
ListResponse
count : int
data : List[NetworkSummaryResponse]
success : bool
LocalCoordinates
entity(coordinate: int): Entity
MultiOmicsCreate
description : Optional[str]
layers : List[OmicsLayerInput]
model_config : ConfigDict
name : str
network_type : str
node_labels : List[str]
organism : str
MultiOmicsCreationWrapper
data
success : bool
data
MultiOmicsCreationResponse
description : str
edge_count : int
layer_names : List[str]
name : str
network_id : int
network_type : str
node_count : int
organism : str
MultiOmicsCreationResult
description : str
edge_count : int
layer_names : List[str]
name : str
network_id : int
network_type : str
node_count : int
organism : str
signature_hash : str
MultiOmicsNetwork
created_at : Optional[str]
description : str
edge_count : int
layers : List[OmicsLayer]
name : str
network_id : int
network_type : str
node_count : int
node_labels : List[str]
organism : str
signature_hash : Optional[str]
MultiOmicsSingleResponse
data
success : bool
data
MultiOmicsResponse
created_at : Optional[str]
description : str
edge_count : int
layers : List[OmicsLayerResponse]
name : str
network_id : int
network_type : str
node_count : int
node_labels : List[str]
organism : str
MultiOmicsSearch
layers : List[OmicsLayerInput]
mode : Literal['coherent', 'independent']
model_config : ConfigDict
node_labels : List[str]
MultiOmicsSearchMatch
edge_count : int
layer_names : List[str]
match_type : str
mode : str
name : str
network_id : int
network_type : str
node_count : int
node_labels : List[str]
organism : str
subgraph_result : str
MultiOmicsSearchMatchResponse
edge_count : int
layer_names : List[str]
match_type : str
mode : str
name : str
network_id : int
network_type : str
node_count : int
node_labels : List[str]
organism : str
MultiOmicsSearchResponse
data : List[MultiOmicsSearchMatchResponse]
success : bool
NativeLibraryError
NativeMultiOmics
path : str
compare(layers_a: np.ndarray, layers_b: np.ndarray, mode: str, strategy: int): Tuple[Optional[str], Optional[str]]
NativeSubgraph
path : str
compare(matrix_a: np.ndarray, matrix_b: np.ndarray): Tuple[Optional[str], Optional[str]]
Network
adjacency_matrix : List[List[int]]
created_at : Optional[str]
description : str
edge_count : int
name : str
network_id : int
network_type : str
node_count : int
node_labels : List[str]
organism : str
signature_array : Optional[List[int]]
signature_hash : Optional[str]
NetworkCreate
adjacency_matrix : List[List[int]]
description : Optional[str]
model_config : ConfigDict
name : str
network_type : str
node_labels : List[str]
organism : str
NetworkCreationResult
description : str
edge_count : int
name : str
network_id : int
network_type : str
node_count : int
organism : str
signature_hash : str
SingleResponse
data
success : bool
data
NetworkResponse
adjacency_matrix : List[List[int]]
created_at : Optional[str]
description : str
edge_count : int
model_config : ConfigDict
name : str
network_id : int
network_type : str
node_count : int
node_labels : List[str]
organism : str
signature_array : Optional[List[int]]
NetworkSearch
adjacency_matrix : List[List[int]]
model_config : ConfigDict
node_labels : List[str]
NetworkSummary
created_at : Optional[str]
edge_count : int
name : str
network_id : int
network_type : str
node_count : int
organism : str
signature_hash : Optional[str]
NetworkSummaryResponse
created_at : Optional[str]
edge_count : int
model_config : ConfigDict
name : str
network_id : int
network_type : str
node_count : int
organism : str
OmicsLayer
adjacency_matrix : List[List[int]]
edge_count : int
layer_name : str
OmicsLayerInput
adjacency_matrix : List[List[int]]
layer_name : str
OmicsLayerResponse
adjacency_matrix : List[List[int]]
edge_count : int
layer_name : str
PairIndex
layer_count : int
add(key: Key, stack: Stack): None
classify(query: Stack, mode: str, layers: Optional[Iterable[int]]): Dict[Key, str]
entry_count(): int
interner_sizes(): List[int]
remove(key: Key): None
search(query: Stack, mode: str, layers: Optional[Iterable[int]], direction: str): IndexSearch
stack(key: Key): Stack
QueryCoordinates
entity(coordinate: int): Entity
RegisterCoordinates
frozen : bool
next_coordinate : int
entity(coordinate: int): Entity
items(): List[Tuple[Entity, int]]
lookup(entity: Entity): Optional[int]
RelationType
arity : int
levels : int
name : str
SearchMatch
edge_count : int
match_type : str
name : str
network_id : int
network_type : str
node_count : int
node_labels : List[str]
organism : str
subgraph_result : str
SearchMatchResponse
edge_count : int
match_type : str
model_config : ConfigDict
name : str
network_id : int
network_type : str
node_count : int
node_labels : List[str]
organism : str
SearchResponse
data : List[SearchMatchResponse]
success : bool
ShimSpec
abi : int
headers : Tuple[str, ...]
label : str
source : Path
Stack
layer_count : int
layers : Tuple[Layer, ...]
length : int
occupancy : int
of(layers: Iterable[Iterable[Iterable[int]]]): 'Stack'
project(layers: Optional[Iterable[int]]): 'Stack'
to_lists(): List[List[List[int]]]
tuples(): Tuple[Tuple[Component, ...], ...]
UniversalStructureRecord
created_at : Optional[str]
description : str
edge_count : int
entity_names : List[str]
length : int
name : str
network_id : int
network_type : str
node_count : int
organism : str
schema_name : str
schema_version : int
signature_hash : Optional[str]
structure
structure
Structure
entities : Mapping[Entity, FrozenSet[str]]
relations : Mapping[str, Mapping[Row, int]]
size : int
build(schema: Schema, entities: 'Mapping[Entity, Iterable[str]] | Iterable[Entity]', relations: Optional[Mapping[str, 'Mapping[Sequence[Entity], int] | Iterable[Sequence[Entity]]']]): 'Structure'
SubgraphConfig
max_workers : Optional[int]
UniversalSearchMatch
edge_count : int
layers : List[str]
match_type : str
mode : str
name : str
network_id : int
network_type : str
node_count : int
organism : str
schema_name : str
schema_version : int
subgraph_result : str
UniversalStructureCreationResult
description : str
edge_count : int
length : int
name : str
network_id : int
network_type : str
node_count : int
organism : str
schema_name : str
schema_version : int
signature_hash : str